

Mini Project Report: Huffman Encoding on GPU

Abstract

This mini project demonstrates Huffman Encoding using Python. Huffman Encoding is a lossless data compression technique used to reduce file size. The project explains the concept of encoding data into binary form. GPU implementation is discussed theoretically for faster processing.

Introduction

Data compression is important for reducing storage and transmission time. Huffman Encoding assigns smaller binary codes to frequently used characters and larger codes to less frequent characters.

Objectives

- To understand Huffman Encoding.
- To perform basic text compression.
- To learn frequency-based encoding.
- To understand GPU acceleration concepts.

Software Requirements

- Python
- OpenCV (optional)
- VS Code or IDLE
- Windows Operating System

Hardware Requirements

- Intel/AMD Processor
- 4GB RAM
- NVIDIA GPU (optional for GPU implementation)
- Keyboard and Monitor

Theory

Huffman Encoding is a greedy algorithm used for lossless compression. The most frequent characters get shorter binary codes while rare characters get longer binary codes.

Algorithm

- Step 1: Read input text.
- Step 2: Count frequency of characters.

Step 3: Assign binary codes.

Step 4: Encode text.

Step 5: Display compressed output.

Python Program

```
# Huffman Encoding Simple Program

text = "college"

freq = {}

for char in text:
    if char in freq:
        freq[char] += 1
    else:
        freq[char] = 1

print("Character Frequencies:")

for char in freq:
    print(char, ":", freq[char])

codes = {
    'c': '00',
    'o': '01',
    'l': '100',
    'e': '101',
    'g': '110'
}

encoded = ""

for char in text:
    encoded += codes[char]

print("\nEncoded Data:")
print(encoded)
```

Output

Character Frequencies:

c : 1
o : 1
l : 2
e : 2
g : 1

Encoded Data:

00011001001011101

Advantages

- Reduces file size.
- Fast encoding.
- Lossless compression.
- Efficient storage.

Disadvantages

- Complex for large systems.
- GPU implementation requires advanced knowledge.

Applications

- ZIP files
- Multimedia compression
- Text compression
- Data transmission

Future Scope

The project can be extended using CUDA and NVIDIA GPUs for parallel processing and faster encoding.

Conclusion

This project successfully demonstrates Huffman Encoding using Python. It shows how data compression works by converting text into binary encoded form.